

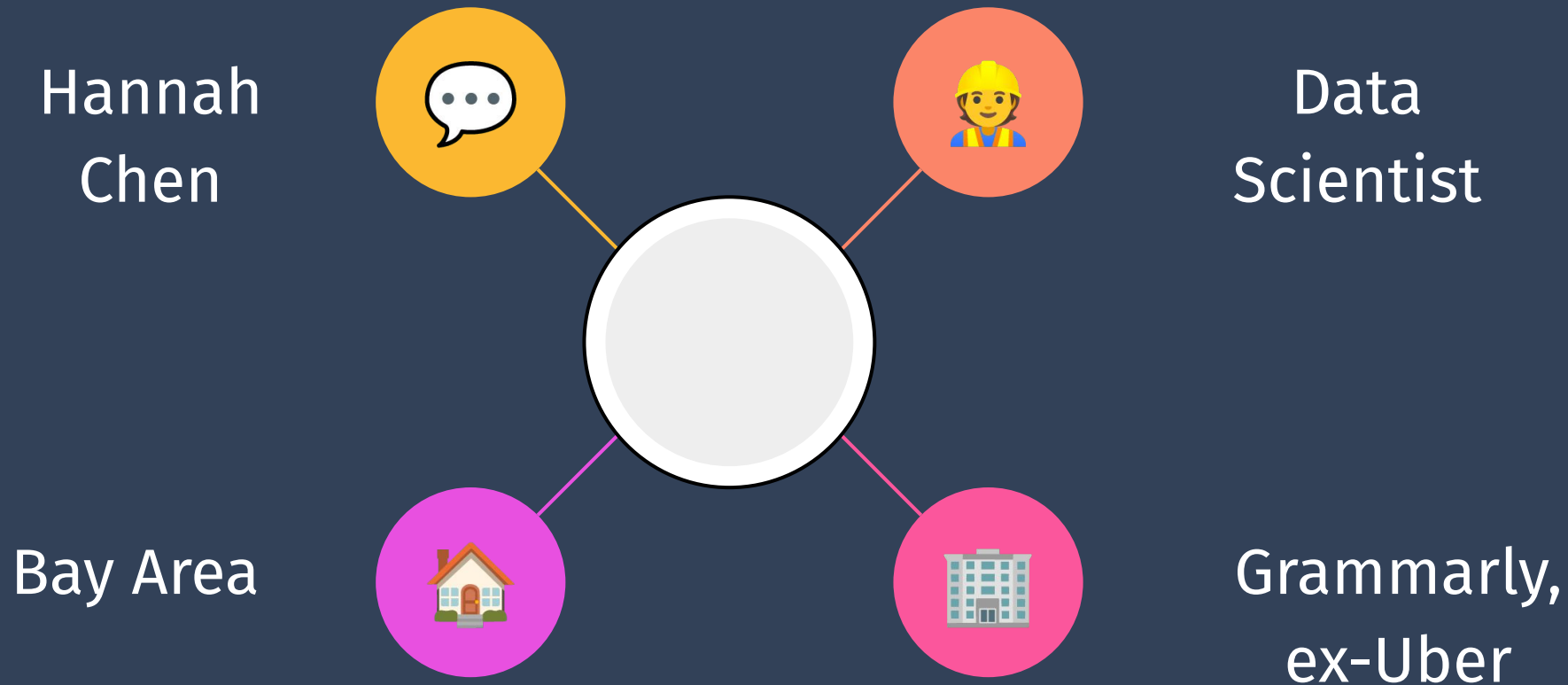
Agentic AI Foundations



**INTERVIEW
KICKSTART**

Instructor

Welcome to today's class, before we begin, pop into the chat:



Structure of Class

1. **Sunday Class - 4 hr (Sunday)**
2. **Assignments - Post class**



Optimize Your Experience

- ✓ Interact with your instructors via Sunday live class.
- ✓ Don't be shy to speak up and get clarifications !



- ✓ Solve your assignments, MCQs and other assessments and get feedback (Saturday review session)
- ✓ Use your resources! It's your experience – what you put in is what you'll get out.

Don't worry!

We are here to help at your every step !

But, how???



Are you using all these IK support features?

- **Uplevel** - contains all the learning resources - videos, MCQs, assignments
- Optional **Post class videos** for additional reference
- **Doubt Clearing/AMA sessions** on Wednesday to clarify all your queries
- **Discord** community for peer learning
- **TAs** to help answer the queries in the live class
- **Support tickets** - Raise them any time you want any clarifications/tech concerns , etc on UpLevel

Success Hacks

- Be prepared for the class by watching the pre class videos
- Attend every class
- Participate in class
- Attempt all the assignments,
- Actively advocate for yourself
- Keep your goal in mind
- Consistency is the key

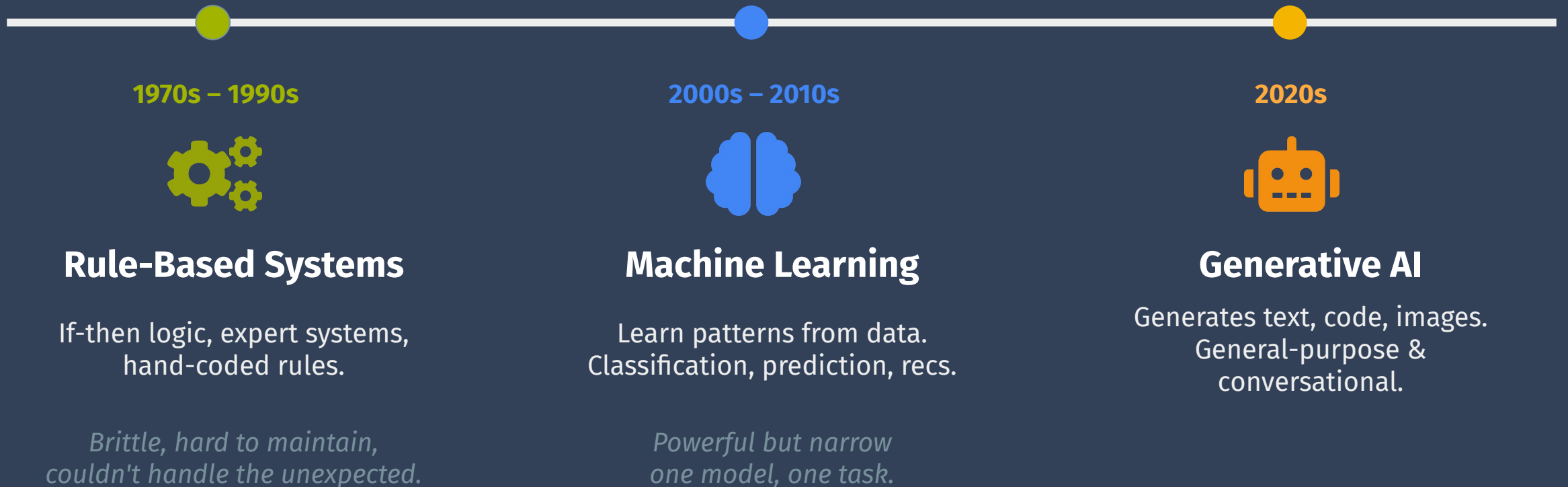
Be patient with yourself !

re

Today's Agenda



Before we start / How we got here



And now GenAI is changing everything...



How

we build software

- AI-assisted coding & code review
- Automated testing & debugging
- Agents handling repetitive tasks

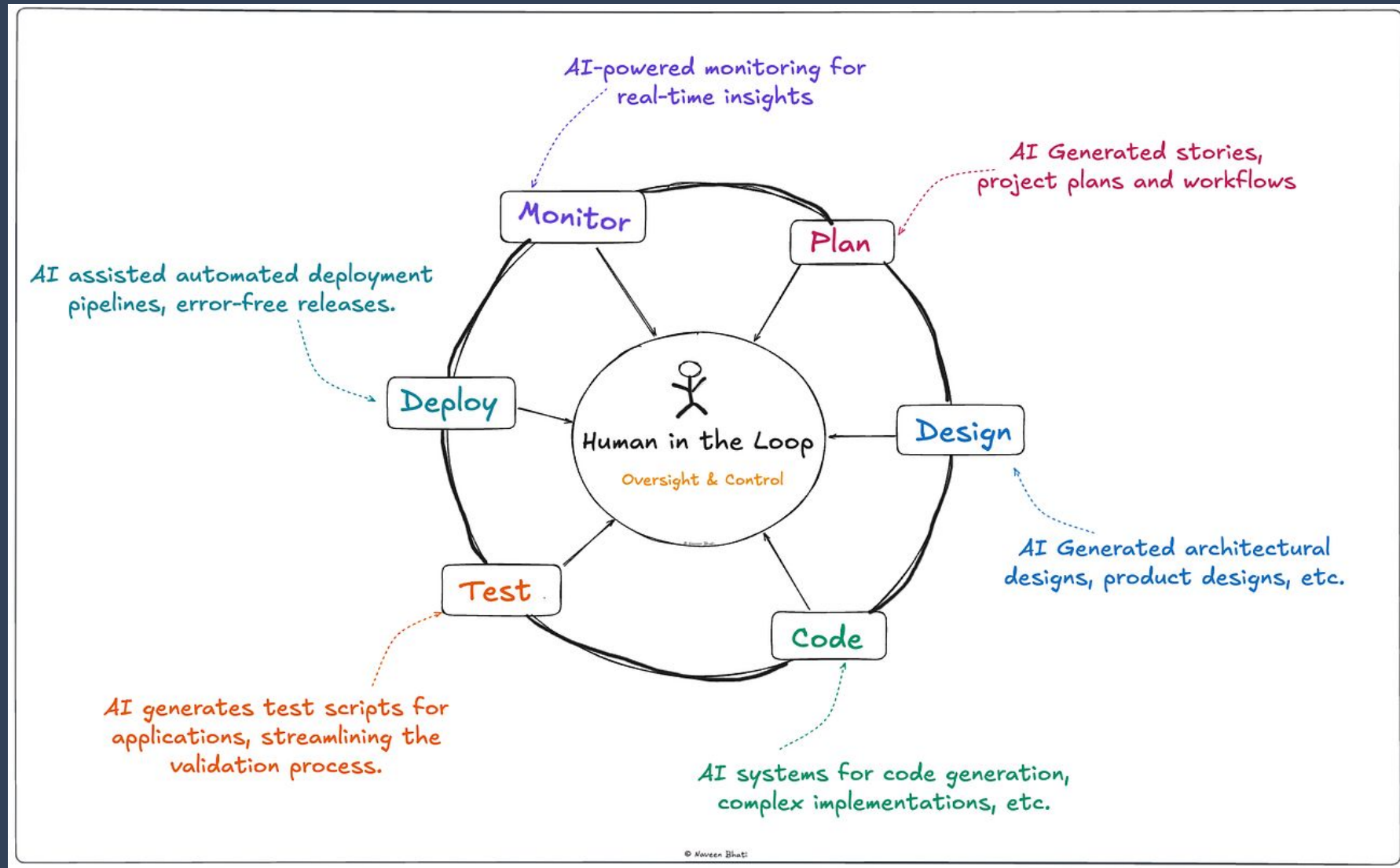


What

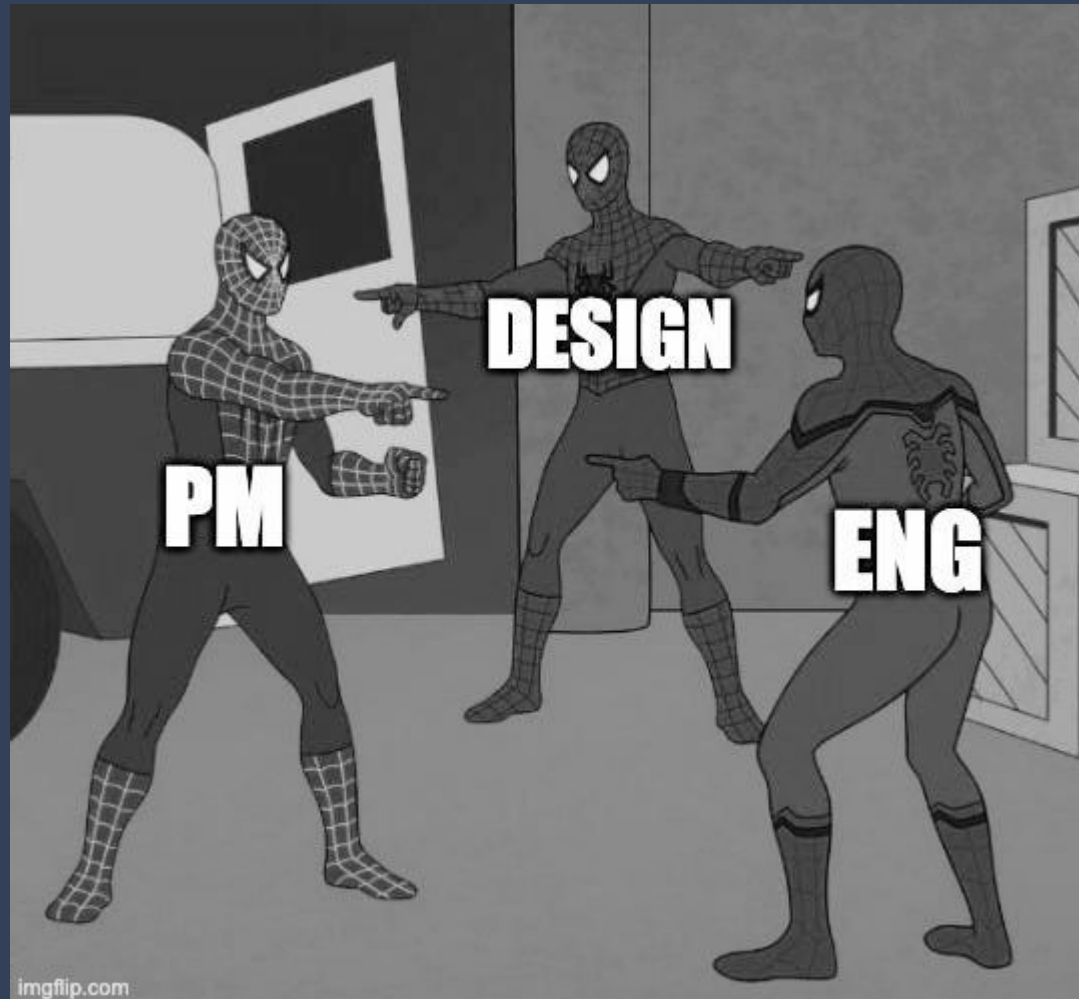
software we build

- AI-native products & features
- Conversational interfaces
- Agent-powered workflows

Entire SDLC becoming AI-Driven



And the lines between roles are blurring



**First let's see what we're going to learn
in this course**

Agentic AI Program Structure

Foundations & Core Agent Setup	Knowledge & Multi-Agent Systems	Conversational Systems & Architecture	Optimization, Personalization & Capstone
0: Pre-Program Foundations • Build AI intuition & AI Exercise Week 0 & 1 1: Agentic Fundamentals (Reflex → Reasoning) • Foundations of GenAI, LLMs & Prompt Engineering, Evolution of AI Agents, Agents & Use cases Routing Pattern • Project: E-commerce Customer Support Agent	2: RAG Knowledge Agents • Embeddings, chunking strategies Vector DBs (FAISS, Chroma) concepts, RAG flow: ingest → index → retrieve → generate Grounded responses & context injection, Retrieval evaluation • Project: Internal Knowledge Assistant Week 2 & 3 3: Multi-Agent Orchestration • Task decomposition, Multi-agent patterns, Cost, determinism, latency tradeoffs, Visual message passing & debugging • Project: AI Research Team	4: Conversational & Multimodal Agents • Memory types + retention, Whisper STT + TTS integration, Multimodal prompts (text/image/audio), Chat/voice interface design • Project: Voice Feedback Agent 5: AI Product Architecture • RAG vs Agents vs Pipelines Data flow diagrams, Latency & throughput constraints • Project: AI System Blueprint Week 4 & 5 	6: Evaluating AI Agents and Optimizing their Performance • LangSmith concepts, Token cost modeling • Project: AI Evaluation Dashboard 7: Personalization & Fine-Tuning of Agents • PEFT/LoRA concepts (simple) Dataset curation, HF Space template training Week 6, 7, 8 & 9 • Project: Personalized Product Advisor 8–9: Capstone Project • No-code orchestration, Strategy deck, KPIs + ROI rationale

By the end of this course, you will be able to:

- 1. **Build and deploy agents**: from simple workflows to multi-agent systems
- 1. **Design agentic systems** with real-world constraints and tradeoffs
- 1. **Integrate agents** into existing tools, tech and workflows
- 1. **Ship production-ready agents** with observability & evaluation
- 2. **Fine-tune and tailor** agentic systems for complex, domain-specific tasks

Now, let's get started

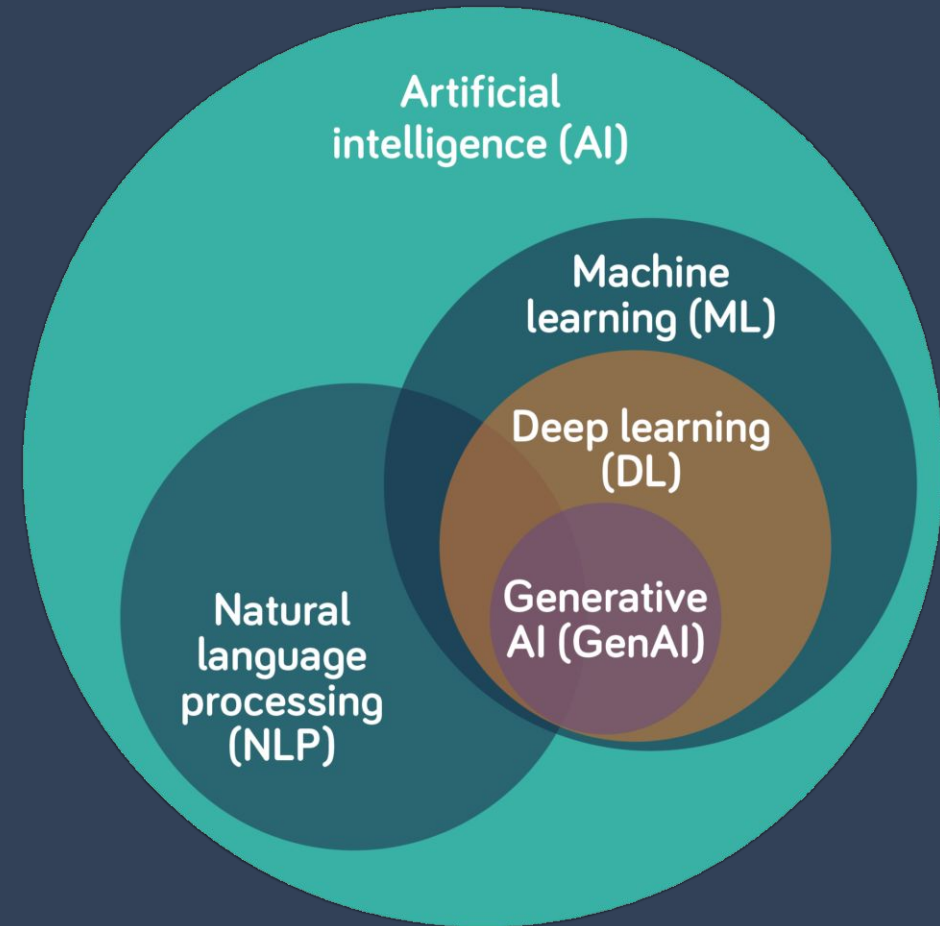
Generative AI (Gen AI)

Generating content with massive AI models

What is Gen AI?



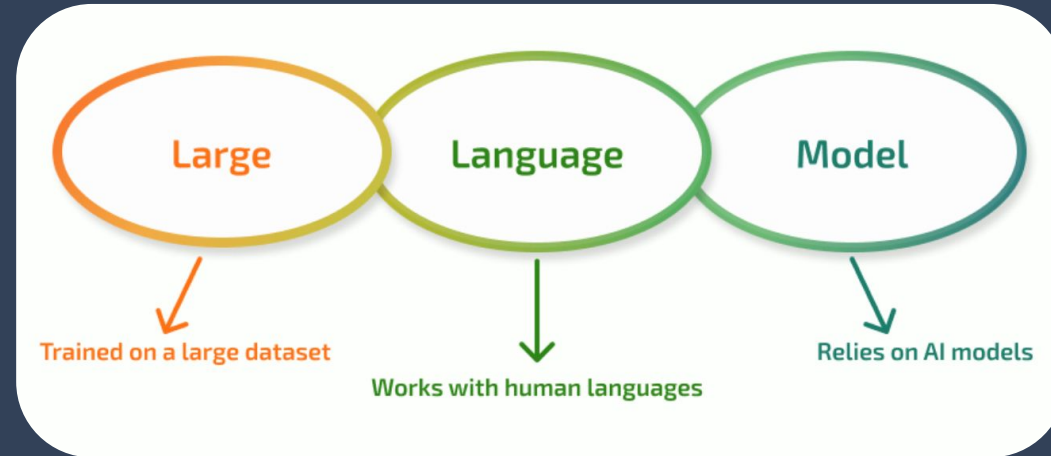
1. Creates **new content** (text, code, images, audio)
2. **Understands** natural language (and multimodal) inputs
3. Powered by large **foundation models** trained on massive datasets





Behind most **Generative AI** system is a
Large Language Model (LLM)

What are LLMs?



- Trained on **massive datasets** to learn patterns, meaning, and reasoning
- **Understand and generate** language and other modalities (text, code, images, audio)
- Built on **transformer architectures that scale** to billions of parameters and long context

If **LLMs** are the **engine**,
prompts are how we **steer** them.



Prompt Engineering

- ❑ **Context** Set the stage: role, goal, plan etc.

"You are a senior SRE reviewing a post-mortem"

- ❑ **Structure** Use delimiters, Markdown, or XML-like tags

Use ## Task, ## Input, ## Output headers to organize your prompt

- ❑ **Constraints** Output format, tone, edge cases

Respond in JSON. Max 200 words. Use only the data provided.

- ❑ **Iterate** Draft, evaluate, review output, refine.

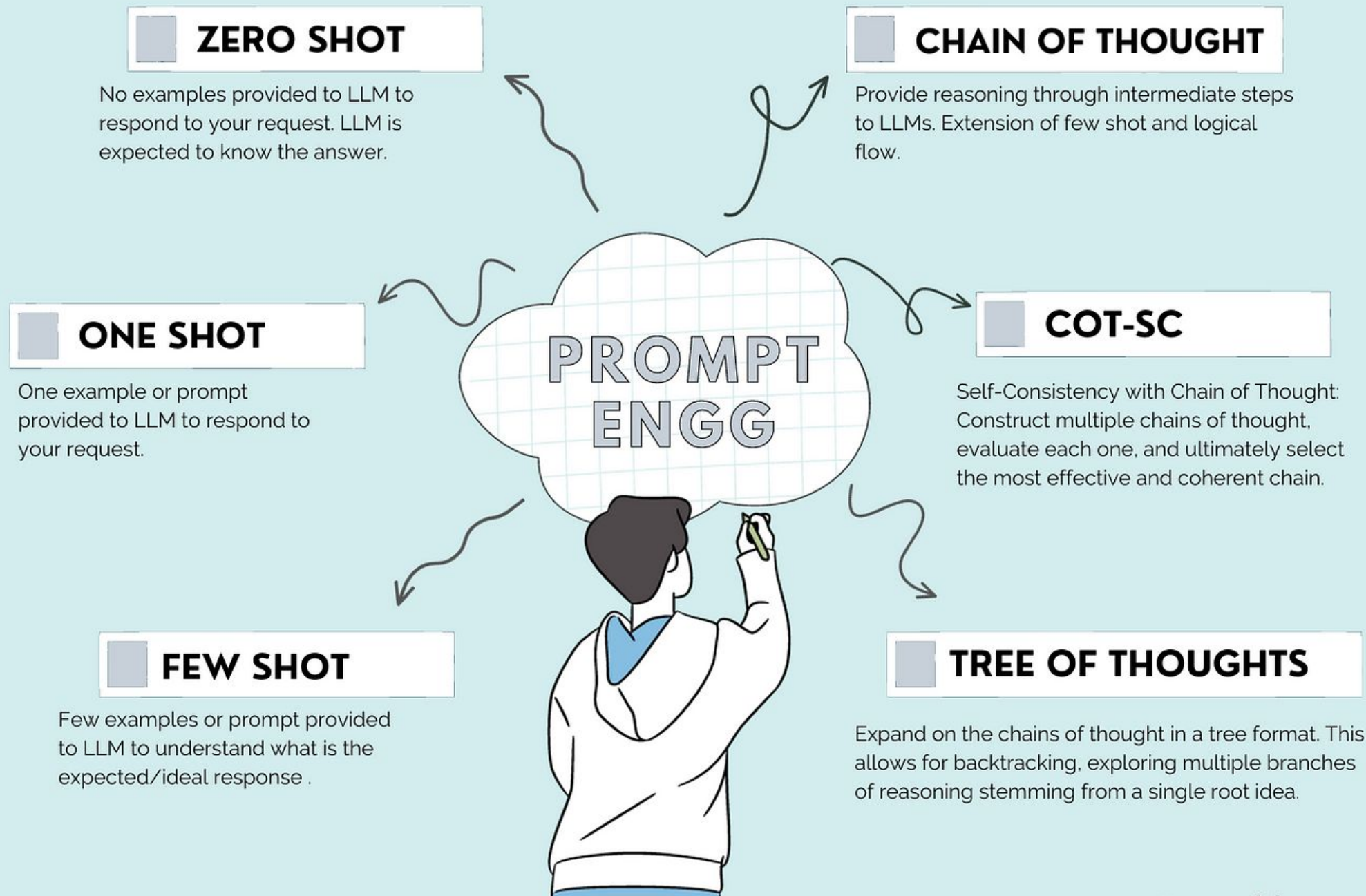
```
## Role
You are a senior SRE reviewing
a post-mortem.
```

```
## Task
Summarize the root cause and
recommend 3 action items.
```

```
## Input
{incident report}
```

```
## Constraints
- Max 200 words
- Respond in JSON
- Use only the data provided
```


TYPES



Limitations of LLMs



No Fresh Knowledge

Trained on a snapshot of the world. Can't access your docs, your databases, or what happened today.



No Memory

Every conversation starts from zero. No recall of past interactions or accumulated context.



No Actions

Generates text, not outcomes. Can't call an API, run a query, or update a ticket.



Hallucinations

Confidently wrong. No built-in way to verify or say "I don't know."

**What if we gave the LLM
tools, memory, and a loop?**

Agent = loop(prompt + tool + memory)



Prompt

Defines the behavior



Tools

Enable actions

APIs, search, code execution, databases



Memory

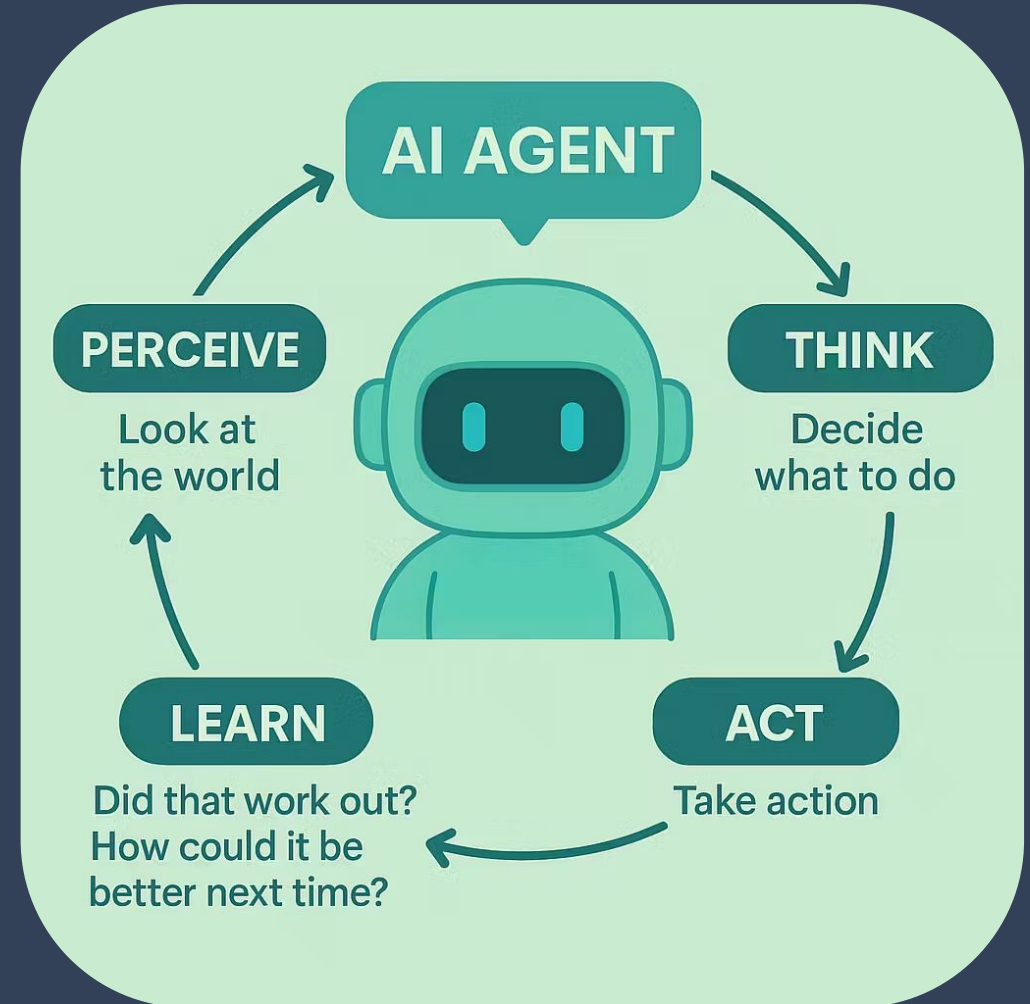
Remembers between turns



Loop

Decides what to do next

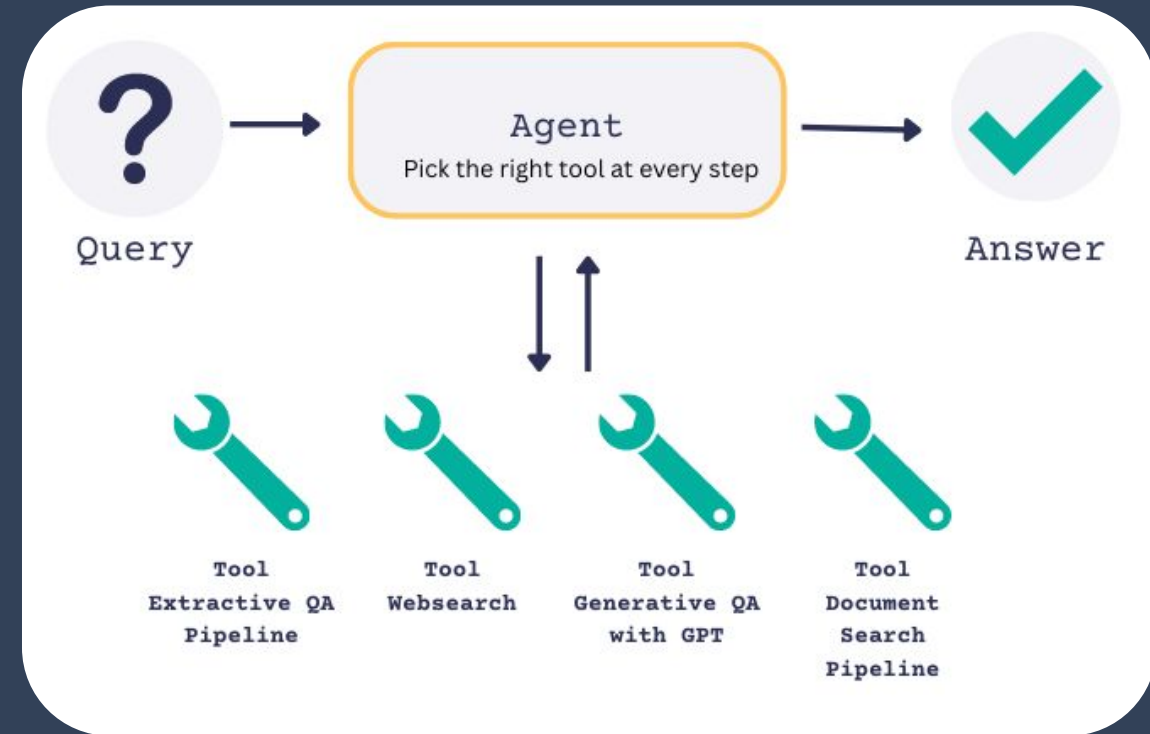
Call a tool, evaluate the result, repeat until done



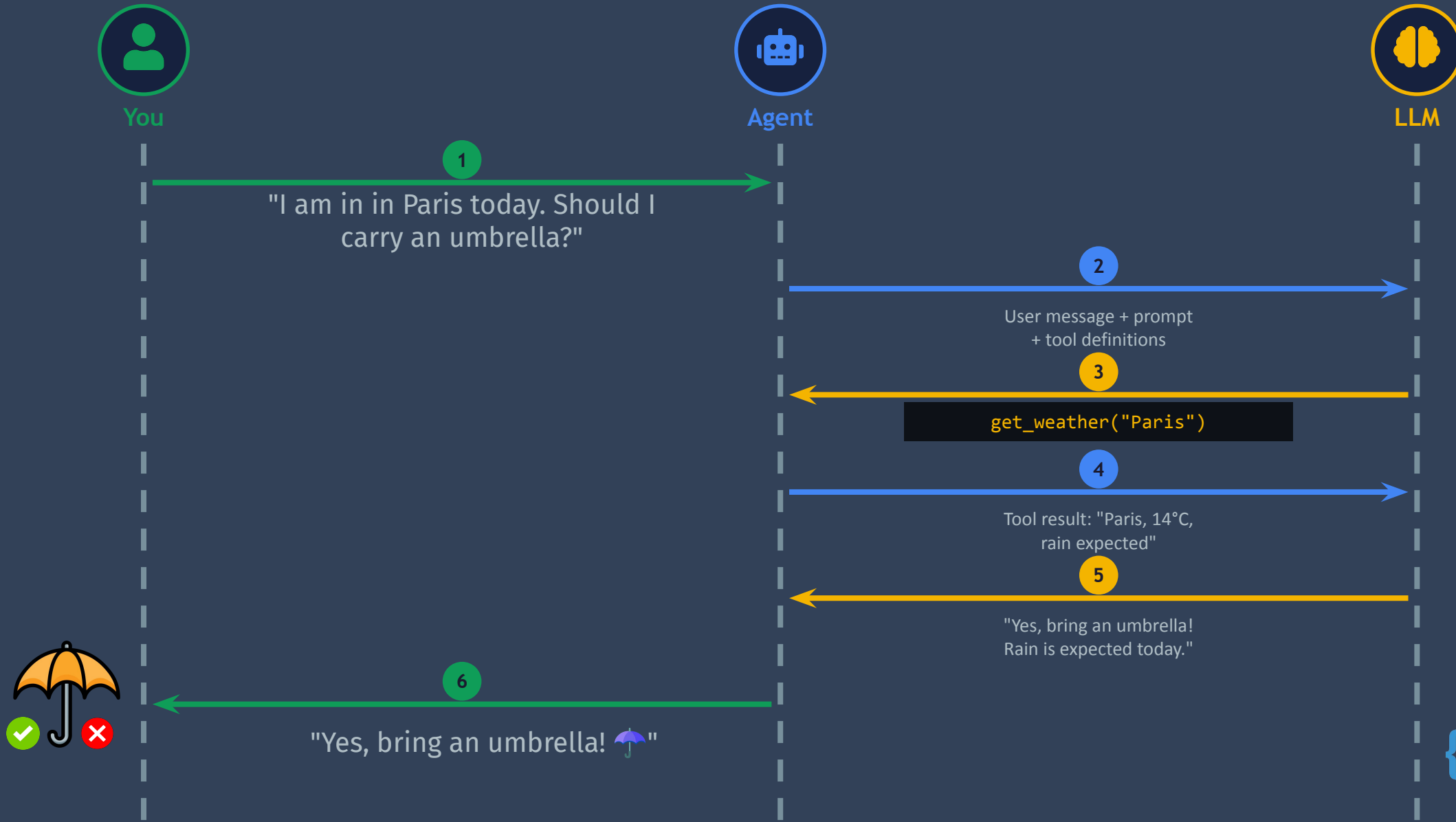
An agent allows LLM to use tools, remember context, and decide what to do next.

Agents with Tools

- Enables the LLM to **retrieve information** from APIs, databases, and knowledge repositories.
- Allows the LLM to **execute actions** in external systems, such as sending emails or querying web services.
- Performs **specialized functions** like code execution, calculations, or document processing based on the task at hand.
- Enhances responses by dynamically selecting the right tool for the given task, improving accuracy and efficiency.

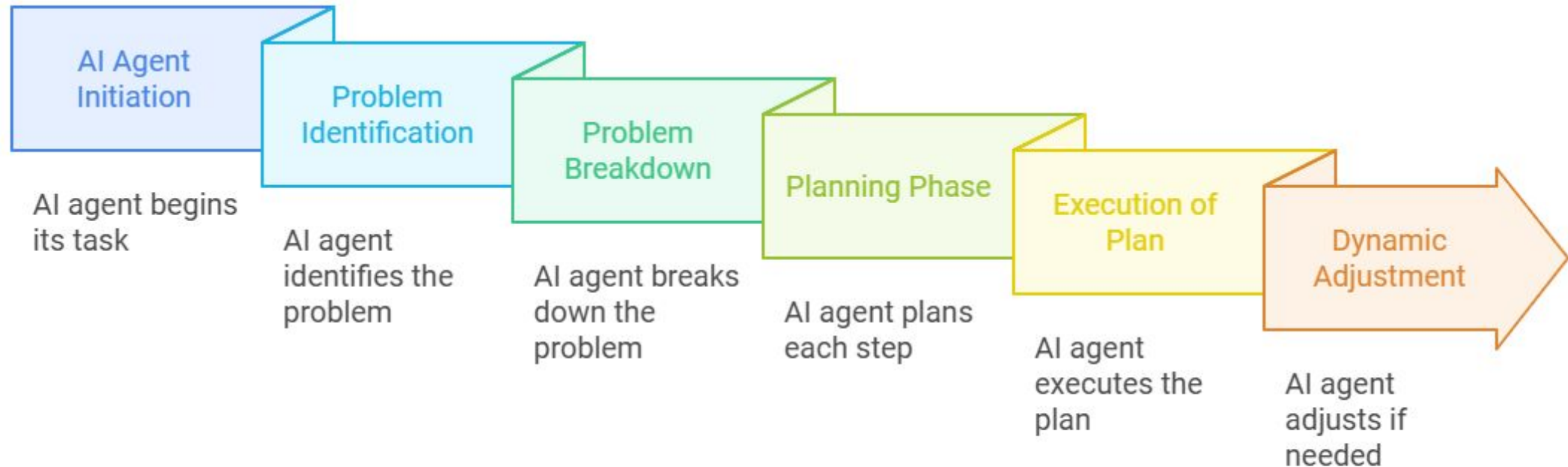


Agent's way of solving problems - Example

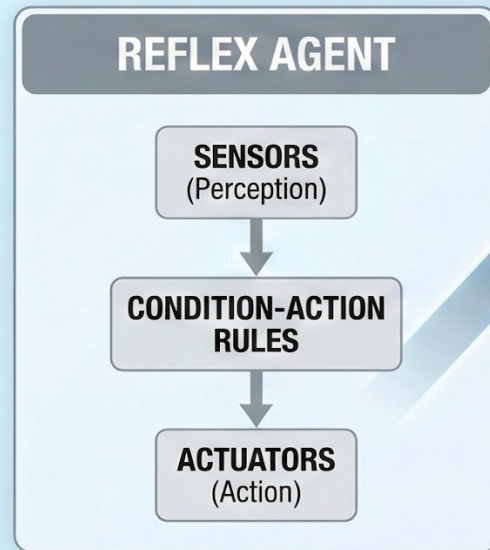


Agent's way of solving problems

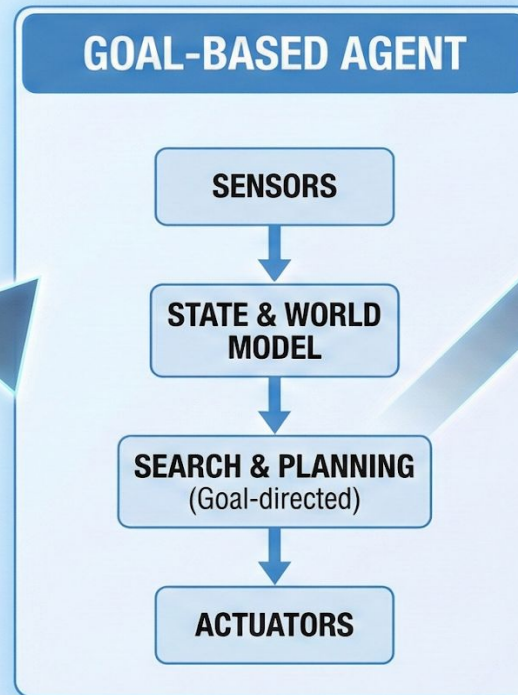
AI Agent Problem-Solving Process



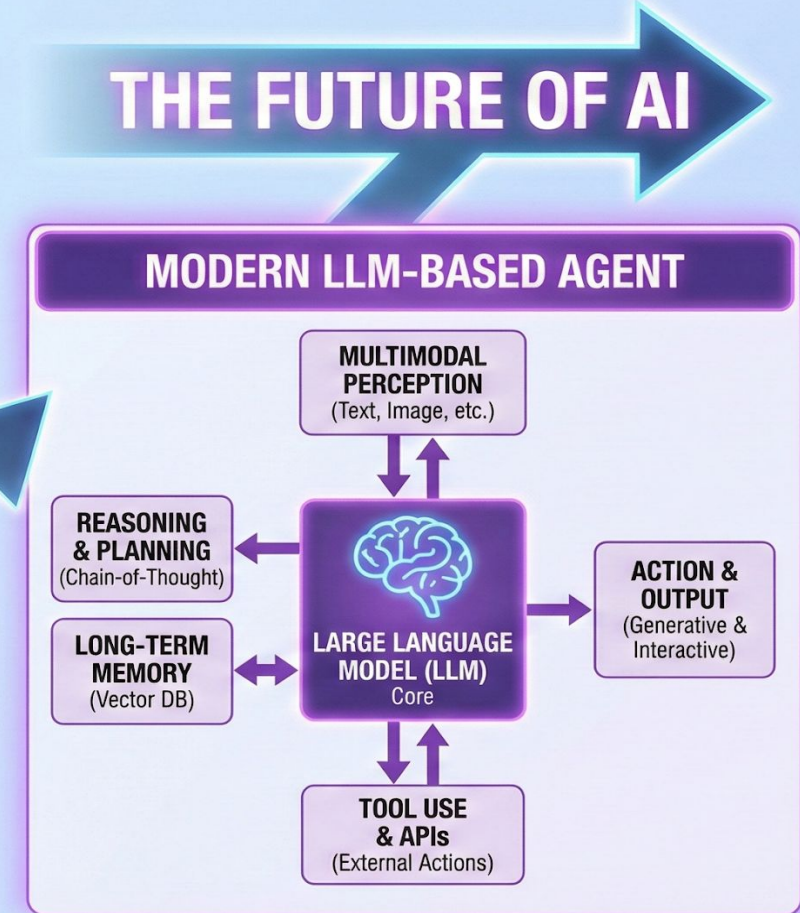
EVOLUTION OF AI AGENTS



Reactive, No Memory,
Simple Tasks



Proactive, Plans Ahead,
Complex Goals



**THE FUTURE: General-Purpose,
Autonomous, Adaptive, Reasoning**

How PMs can leverage Agents

From signals to decisions: scale product thinking with agents

1



Automated discovery

Continuously identifying user pain points from multiple signals.

Example: Agent scans reviews, support tickets, and analytics to flag a rising checkout issue.

2



Intelligent prioritization

Continuously identifying user pain points from multiple signals.

Example: Agent ranks feature ideas using impact, effort, and revenue signals.

3



Experiment Orchestration

Designing and running experiments end-to-end.

Example: Agent sets up an A/B test, tracks results, and summarizes learnings.

4



Customer Operations Automation

Handling repetitive customer-facing decisions automatically.

Example: Agent detects negative reviews and drafts contextual support responses.

5



Spec & Requirement Generation

Turning decisions into execution-ready artifacts.

Example: Agent converts decisions into a PRD, user stories, and acceptance criteria.

6



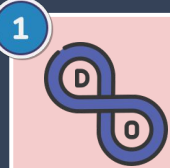
Stakeholder Communication

Keeping everyone aligned without manual effort.

Example: Agent generates a weekly product update for leadership.

How TPMs can leverage Agents

From orchestration to reliability: scale engineering workflows with agents



1 CI/CD Orchestration

Automating release workflows and enforcing quality gates.

Example: Agent blocks a release when test coverage drops below threshold and notifies owners.



2 Incident Triage

Making sense of alerts and guiding response actions.

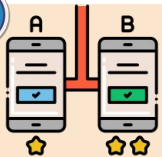
Example: Agent analyzes PagerDuty alerts, identifies the likely root cause, and suggests next steps.



3 System Integration

Coordinating workflows across multiple services and tools.

Example: Agent syncs Jira, GitHub, and Slack during a production rollout.



4 Test Generation

Creating tests directly from specs and code changes.

Example: Agent generates regression tests from a new API contract.



5 Cost & Performance Optimization

Monitoring usage patterns and triggering efficiency actions.

Example: Agent detects a cost spike and recommends scaling or caching changes.



6 Security and compliance checks

Continuously enforcing security and policy requirements.

Example: Agent flags a misconfigured IAM role and generates remediation steps.



Knowledge Check: Agent Fundamentals

1. LLMs are best described to engineering teams as:

- A** Databases for unstructured data
- B** Probabilistic compression engines
- C** Deterministic search algorithms

Knowledge Check: Agent Fundamentals

1. LLMs are best described to engineering teams as:

Answer: B. Probabilistic compression engines

LLMs learn statistical patterns in massive amounts of text and generate outputs by predicting the most likely next token. In doing so, they effectively compress knowledge into probabilities, rather than storing exact facts like databases or following fixed, deterministic search rules.

Knowledge Check: Agent Fundamentals

2. What is the key distinction of a Reflex Agent compared to a Reasoning Agent?

- A** It relies on Vector DB for all answers
- B** It uses hardcoded, if-then logic (fast, cheap)
- C** It can execute arbitrary code functions

Knowledge Check: Agent Fundamentals

2. What is the key distinction of a Reflex Agent compared to a Reasoning Agent?

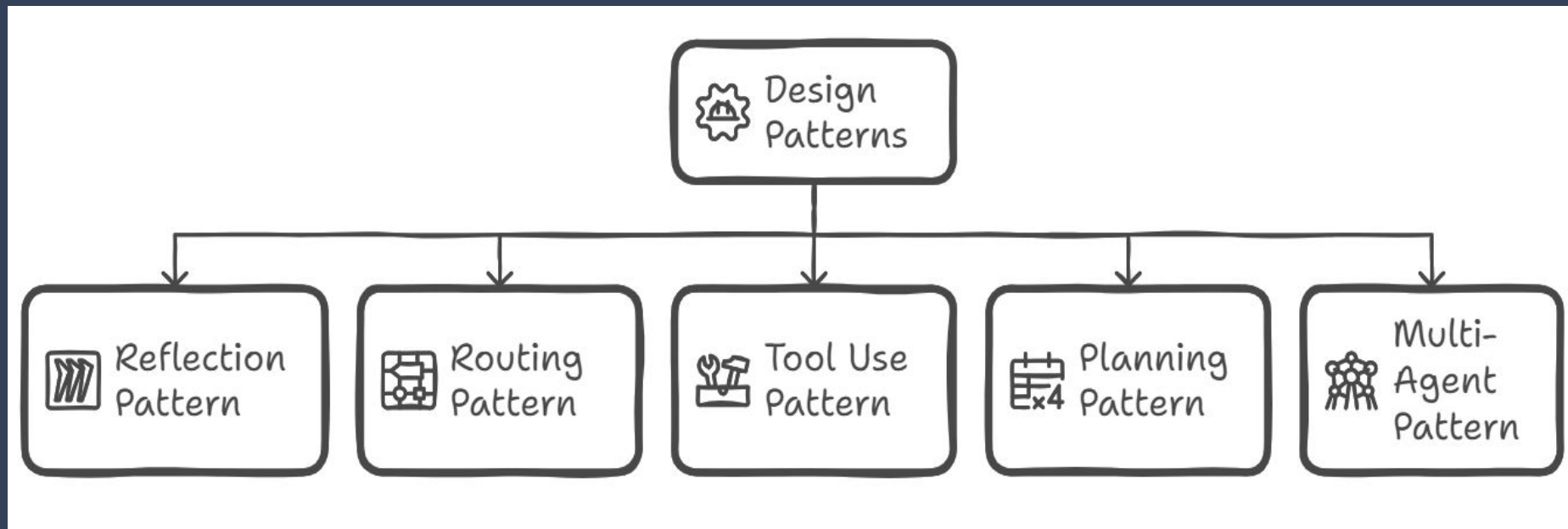
Answer: B. It uses hardcoded, if-then logic (fast, cheap)

Reflex agents operate using predefined condition–action rules and respond directly to inputs without planning or reasoning. This makes them faster and cheaper than reasoning agents, which require additional computation to analyze, plan, and decompose tasks.

You've learned what agents can do.
Now let's learn **how to design** them for real-world applications.

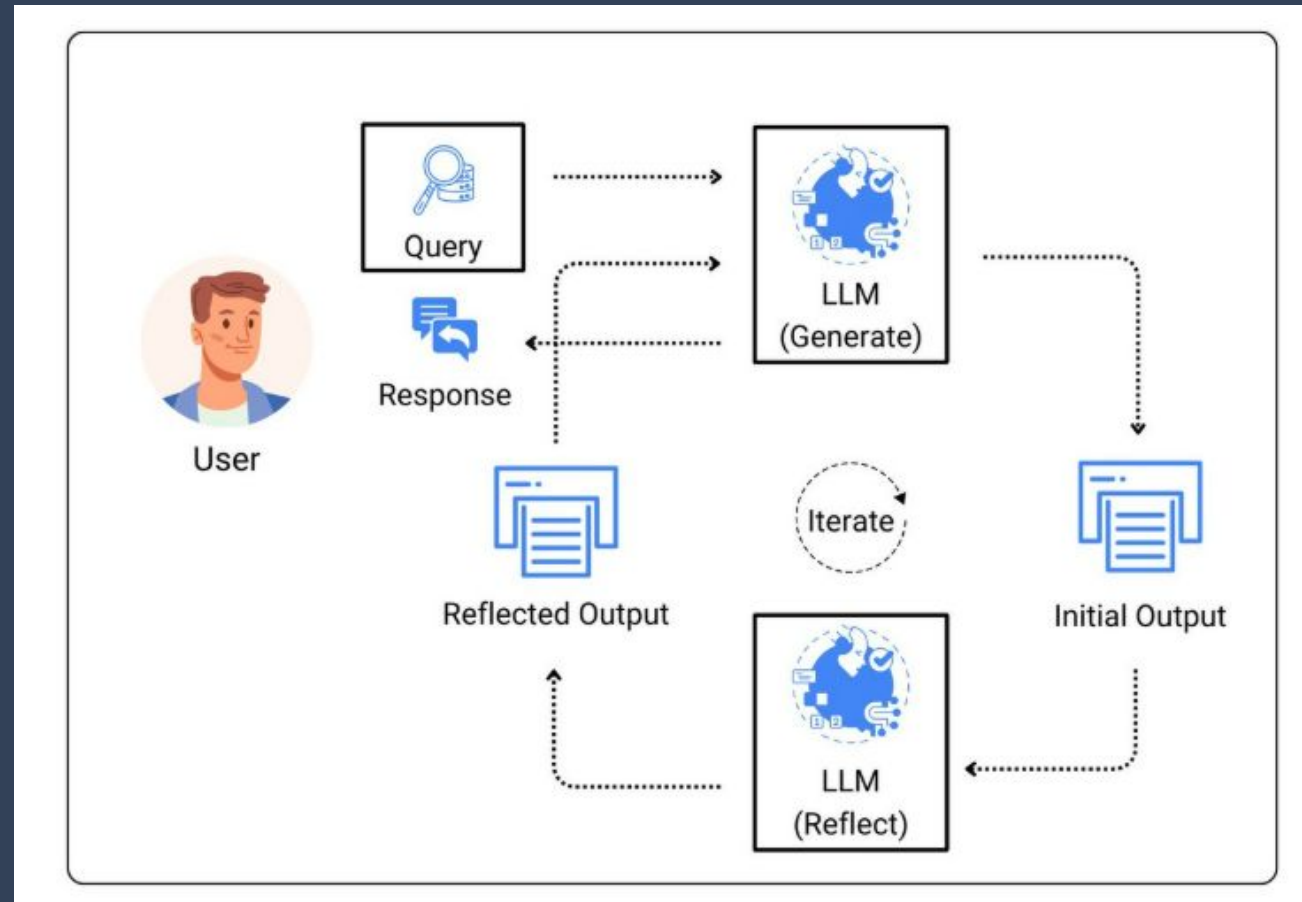
Agentic design Pattern

- Agentic Design Patterns are the **building blocks for LLM systems** that go beyond single prompt-response interactions.
- They use composable structures to orchestrate reasoning, decisions, tools, and control flow.
- By replacing hard-coded logic with system structure, these patterns enable reliable adaptation and scaling—from simple assistants to autonomous, goal-driven agents.



Reflection Pattern

The agent reflects on its actions,
decisions and reasoning,
It critiques and refines
its own outputs
before finalizing a response.



Routing — a core pattern for LLM-based agents.

More Agentic Design Patterns coming in later weeks.

Routing

Routing is the mechanism that decides **what the agent should do next** at runtime.

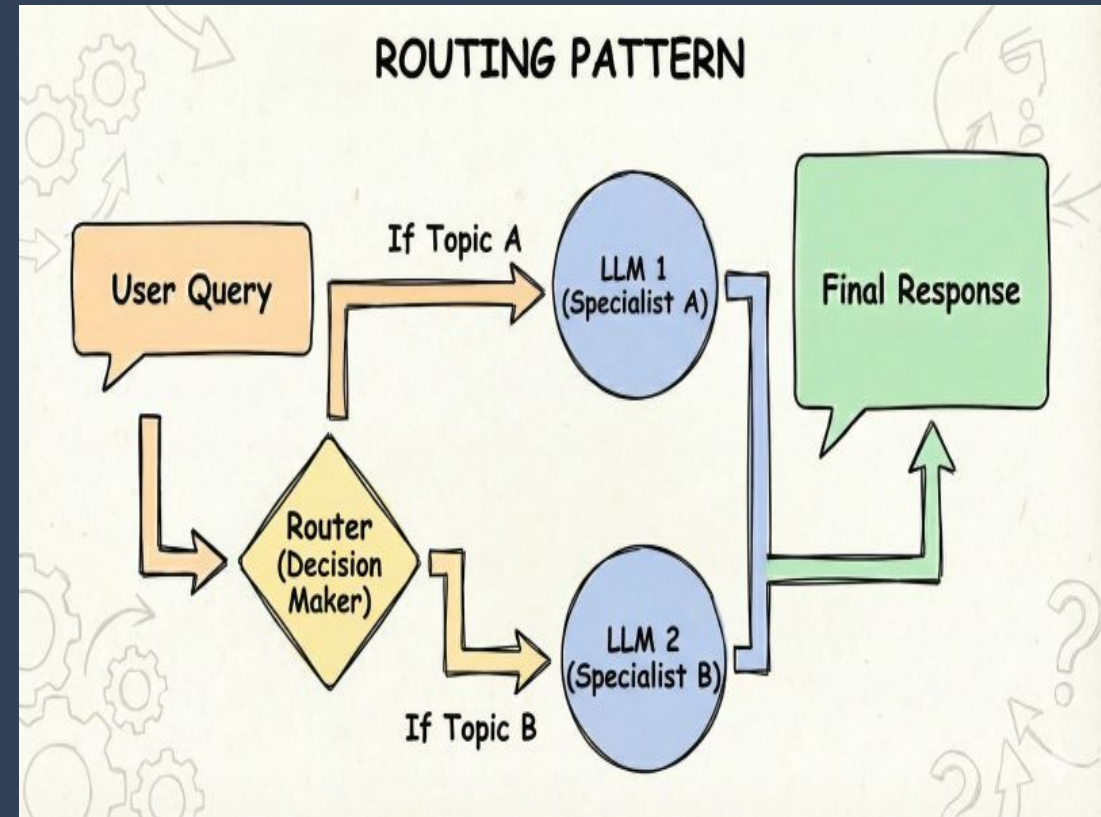
Instead of following a fixed sequence of steps, the agent:

- Evaluates current input
- Considers system state and past outcomes
- Chooses the most appropriate next action

Example:

- IF review is negative → generate empathetic email
- IF review is positive/neutral → take no action

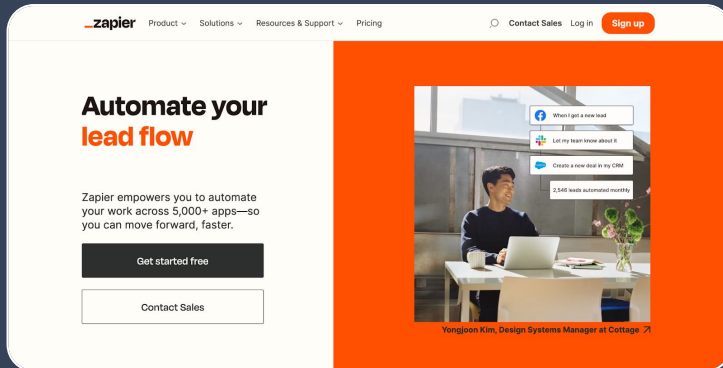
Without routing, an agent is just a linear workflow.



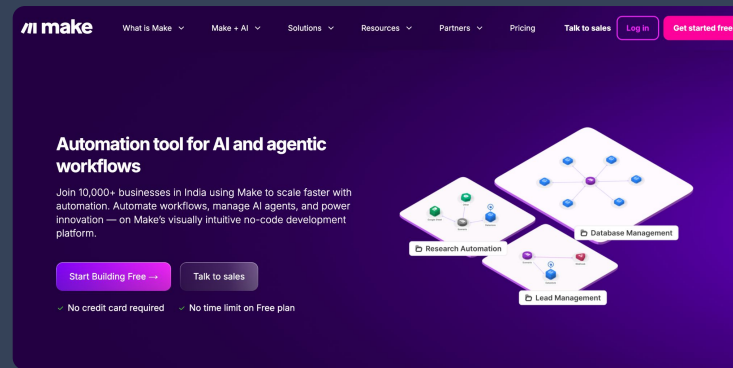
**Let's see how the Routing Pattern
is implemented in our Agent
Time To Build Agent**

No-code Agent Builder Tools

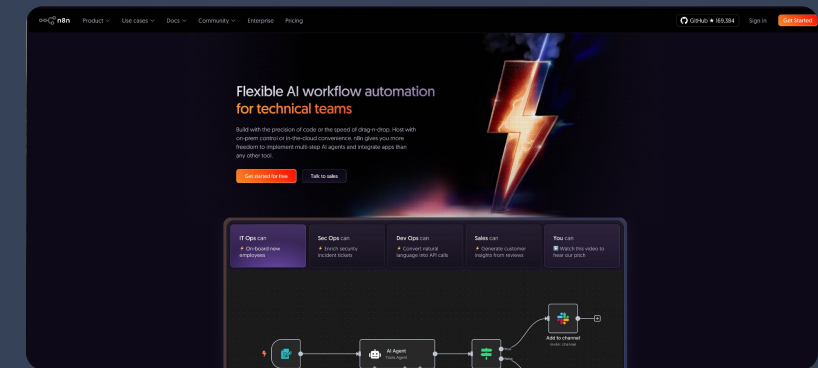
zapier



make



n8n



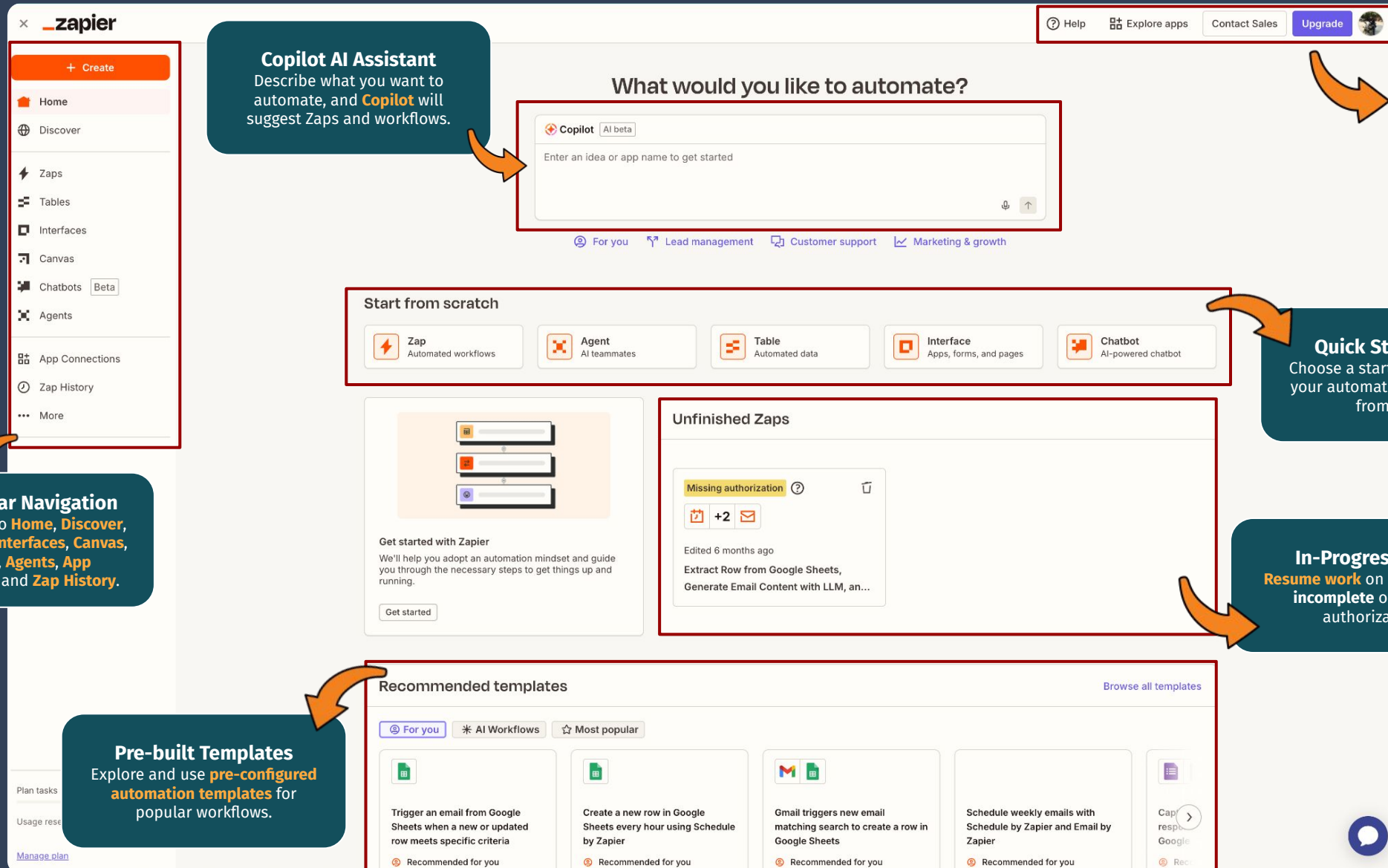
For today's session, we'll use **Zapier** to build and demonstrate agents



A pioneer in building action driven
agents without code

Interface Tour

Home Page



Copilot AI Assistant
Describe what you want to automate, and **Copilot** will suggest Zaps and workflows.

What would you like to automate?

Copilot AI beta

Enter an idea or app name to get started

For you Lead management Customer support Marketing & growth

Start from scratch

- Zap**
Automated workflows
- Agent**
AI teammates
- Table**
Automated data
- Interface**
Apps, forms, and pages
- Chatbot**
AI-powered chatbot

Get started with Zapier
We'll help you adopt an automation mindset and guide you through the necessary steps to get things up and running.

Get started

Unfinished Zaps

Missing authorization ⓘ

+2

Edited 6 months ago

Extract Row from Google Sheets, Generate Email Content with LLM, an...

Recommended templates [Browse all templates](#)

For you AI Workflows Most popular

- Trigger an email from Google Sheets when a new or updated row meets specific criteria
Recommended for you
- Create a new row in Google Sheets every hour using Schedule by Zapier
Recommended for you
- Gmail triggers new email matching search to create a row in Google Sheets
Recommended for you
- Schedule weekly emails with Schedule by Zapier and Email by Zapier
Recommended for you

Top Menu & Profile
Access **help** resources, **explore** more apps, **contact sales**, **upgrade your plan**, and **manage** your profile.

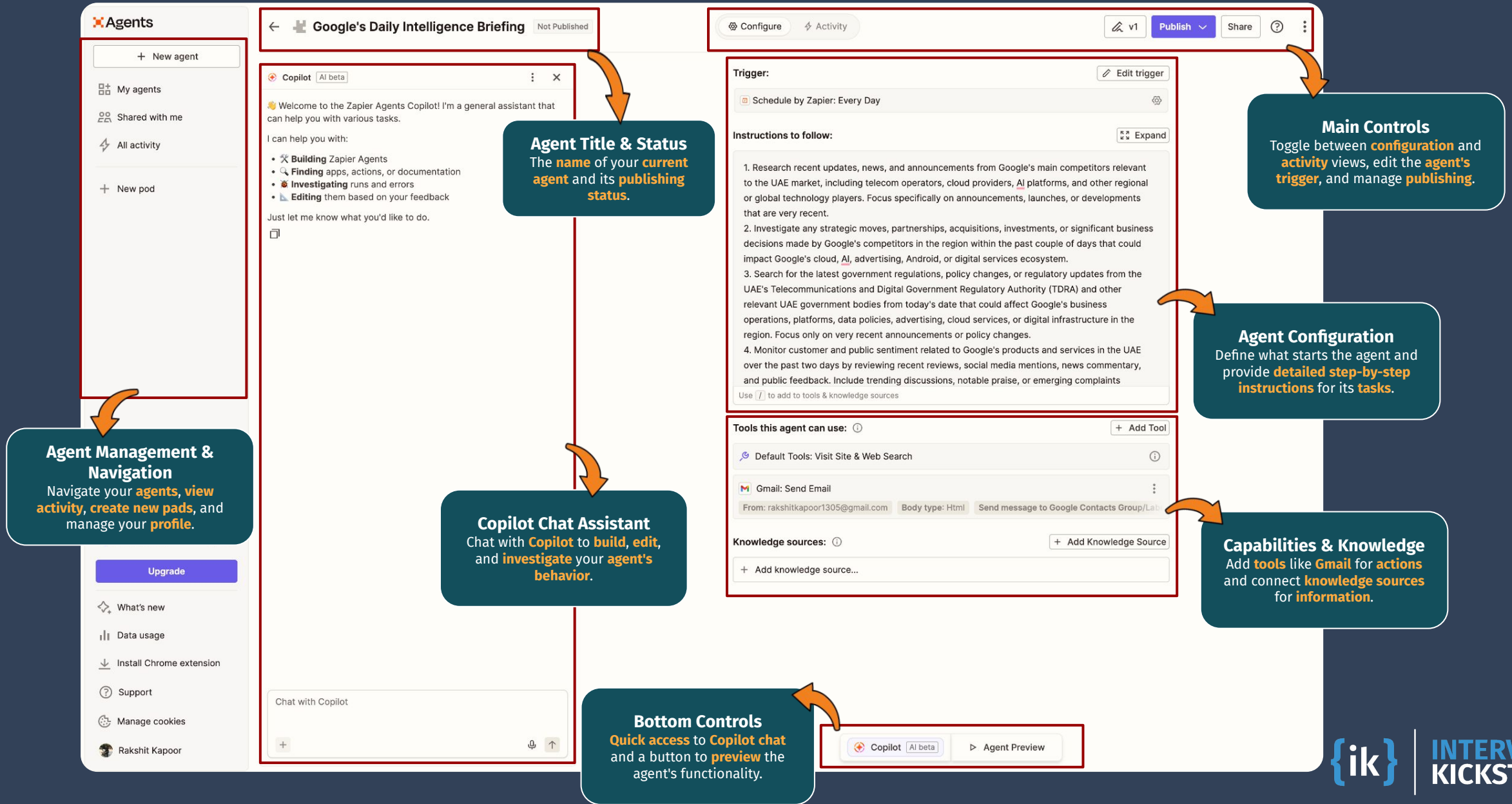
Quick Start Options
Choose a starting point to build your automation or application from **scratch**.

In-Progress Work
Resume work on Zaps that are **incomplete** or missing authorization.

Left Sidebar Navigation
Quick access to **Home**, **Discover**, **Zaps**, **Tables**, **Interfaces**, **Canvas**, **Chatbots**, **Agents**, **App Connections**, and **Zap History**.

Pre-built Templates
Explore and use **pre-configured automation templates** for popular workflows.

Agent Page



Zapier Integrations in Action

Gmail

Automatically prioritize support emails, draft replies for negative queries, and tag or escalate important conversations.



Jira

Convert bugs, incidents, or customer complaints directly into Jira tickets with context attached.

Google Sheets

Track reviews, leads, or metrics in real time and update rows automatically based on agent decisions.



Notion

Auto-update docs, meeting notes, or SOPs based on decisions and outcomes generated by the agent.

Slack

Notify teams instantly when incidents occur, reviews turn negative, or approvals are required.



Calendly

Schedule meetings automatically when conditions are met, like escalation-required reviews or high-priority leads.

Problem Statement

Every ecommerce team drowns in customer reviews and misses critical signals hidden inside them.



The Issue

- Hundreds of reviews land daily across products
- Each review needs to be read, classified, and responded to.
- Support teams spend time on deciding what to do, not just replying



What the Agent Does

- Continuously monitors incoming reviews in real time.
- Automatically analyzes sentiment and intent.
- Decides whether a response is required
- Drafts a professional, empathetic reply for negative reviews

Zapier E Commerce Customer Support Agent Demo

Let us Build an agent using Zapier

Q&A Time

This is your time to clear doubts

- So far, do you have any questions?
- Feel free to drop them in the chat or ask directly
- No question is too small, let's keep the flow clear before we move on

Live Zoom Poll



We are launching a poll on Zoom. Please take 2 minutes to fill out the form.

Week 0 - Mini Project

Build → Document → Publish (Every Agent)

For every agent built in this course:

- Package the project with a clear, structured README
- Upload it to your GitHub portfolio
- Treat each agent as a standalone, portfolio-ready artifact

This workflow will be applied consistently in every class as agents are built and published.

**Thank
you**